

DEVELOPER GUIDE · VERSION 1.0 · APRIL 2026

Comprehensive Guide for *Generative AI*

A complete guide for Python & Java developers — from beginner to intermediate — covering LLMs, Tokens, Embeddings, and Prompt Engineering.

 Target: Python & Java Devs Level: Beginner → Intermediate Last Updated: April 2026

TABLE OF CONTENTS

01

What is Generative AI & LLMs

Defining GenAI · What are LLMs · Training · Popular Models · Capabilities & Limitations · Real-World Examples

02

Tokens, Embeddings & Vectors

Tokens · Tokenization Methods · Token Limits · Embeddings · Similarity Metrics · Practical Applications

03

Prompt Engineering

What is Prompting · Zero-Shot · Few-Shot · Chain-of-Thought · Best Practices · Real-World Examples

SECTION 01

What is *Generative AI* & Large Language Models

Understand the foundations: what generative AI actually is, how LLMs are built, and why they've become transformative for developers.



1.1 Defining Generative AI

AI systems that **create new, original content** — not just classify or analyze existing data.

Generative AI (GenAI) refers to a category of artificial intelligence systems that can create new, original content rather than just analyzing or classifying existing data. Unlike traditional AI models that might tell you whether a picture contains a cat or a dog, **generative AI can draw a new cat that has never existed before.**

Key Characteristics of Generative AI



Creation

Produces novel outputs. Example:
Writing a poem about Mars.



Pattern Learning

Learns underlying patterns — grammar, style, structure — from training data.



Probabilistic

Same input can produce different outputs. "Tell me a joke" → different joke each time.



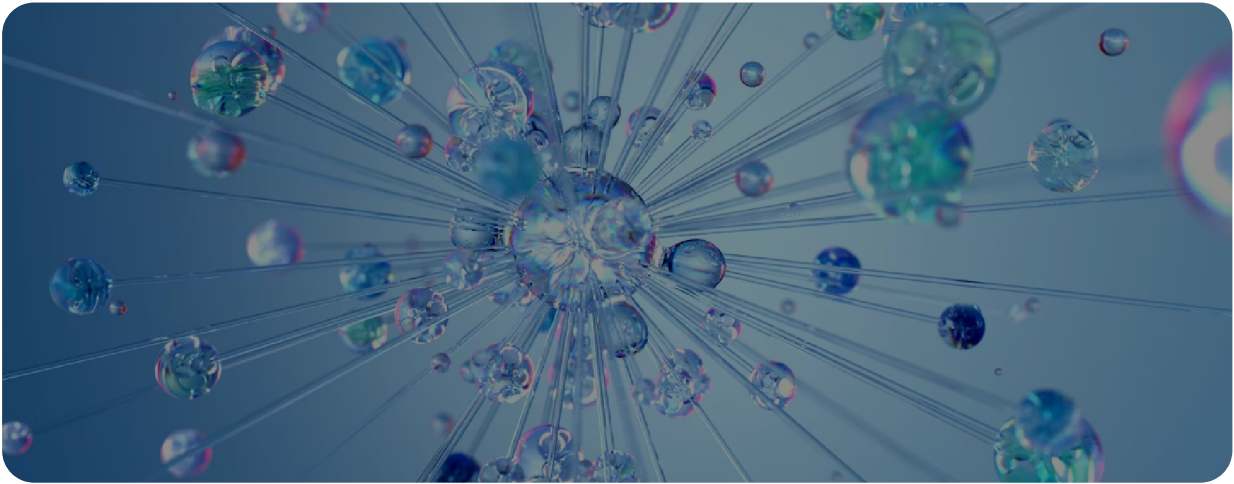
Multimodal

Can work with text, images, audio, video — generate image from text description.

Types of Generative AI

TYPE	INPUT	OUTPUT	EXAMPLES
Text-to-Text	Text	Text	ChatGPT Claude Gemini
Text-to-Image	Text	Image	DALL-E Midjourney
Text-to-Audio	Text	Speech / Music	ElevenLabs Suno
Text-to-Video	Text	Video	Sora Runway Gen-2
Code-to-Code	Code	Code	GitHub Copilot

1.2 What Are Large Language Models (LLMs)?



LLMs contain billions of learned parameters that capture language patterns from vast training data.

Large Language Models are a specific type of generative AI focused on understanding and generating human language. The term *"large"* refers to two things:

~1.7T

Parameters in GPT-4

Trillions

Words in Training Data

<1s

Response Time (small models)

 SIMPLE ANALOGY

Think of an LLM as a *supercharged autocomplete engine* that has read almost everything on the internet. When you give it a prompt, it predicts what words should come next based on patterns learned during training. You type: "*The capital of France is...*" → LLM predicts: "*...Paris*"

1.3 How LLMs Are Trained

LLM training happens in three main phases:

1

Pre-training (Self-Supervised Learning)

The model reads massive amounts of text — internet, books, papers. Learning task: predict the next word. Data scale: hundreds of billions to trillions of words. Duration: weeks to months on thousands of GPUs. Output: a "base model" that can complete sentences.

2

Supervised Fine-Tuning (SFT)

Human labelers create question-answer pairs. The model learns to follow instructions and answer questions. Data scale: tens of thousands to millions of examples. Output: a model that responds helpfully to prompts.

3

Reinforcement Learning from Human Feedback (RLHF)

Humans rank multiple model outputs from best to worst. The model learns what humans prefer — helpful, harmless, honest. Output: a model that aligns with human values and preferences.

1.4 Popular LLMs Today

MODEL	COMPANY	SPECIAL STRENGTHS	ACCESS
GPT-4 / GPT-4 Turbo	OpenAI	General purpose, coding, reasoning	API ChatGPT Plus
Claude 3	Anthropic	Long context (200K tokens), safety	API Claude.ai
Gemini Ultra/Pro	Google	Multimodal (text, image, video)	API AI Studio
Llama 3 (8B, 70B)	Meta	Open source, customizable	Hugging Face
Mistral (7B, Large)	Mistral AI	Efficient, open weights	API
DeepSeek V3	DeepSeek	Cost-efficient, 671B params	API

1.5 Capabilities & Limitations



LIMITATION	EXPLANATION	EXAMPLE FAILURE
Hallucination	Confidently stating false information	"The Eiffel Tower is in Berlin"
Outdated Knowledge	Training data has a cutoff date	Doesn't know events after April 2024 (GPT-4)
Math & Logic	Simple arithmetic mistakes	$7,342 \times 9,871$ — often wrong
Counting	Cannot count letters reliably	How many 'r's in "strawberry"? (says 2, actually 3)
Context Limit	Forgets early parts of long conversations	200K token limit, performance degrades

SECTION 02

Tokens, Embeddings & *Vector Representations*

Under the hood of every LLM: how text becomes numbers, and how those numbers capture meaning.

2.1 What Are Tokens?

Tokens are the smallest units of text that an LLM processes — chunks of characters representing common sequences.

LLMs do not read text letter-by-letter. They break text into **tokens** — chunks of characters that represent common sequences. Understanding tokens is essential for controlling costs and performance.

Examples of Tokenization

TEXT	TOKENS	BREAKDOWN
"Hello"	1	["Hello"]
"Hello world"	2	["Hello", " world"]
"unbelievable"	3	["un", "believe", "able"]
"I love pizza!"	5	["I", " love", " pizza", " !"]

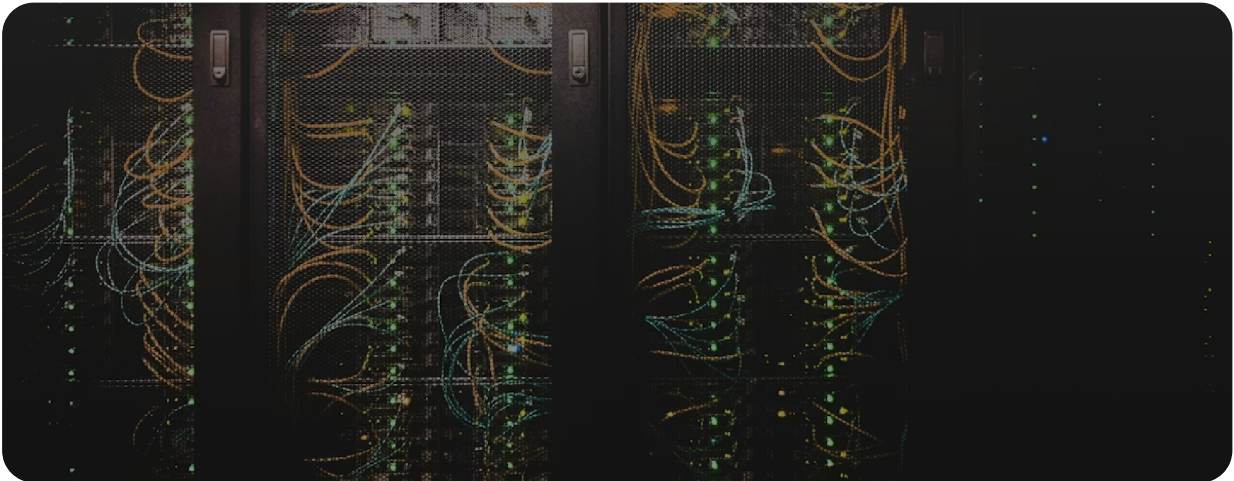
RULES OF THUMB

English text $\approx$ *1.3 tokens per word* · Code $\approx$ *1.5–2 tokens per word* ·
Chinese/Japanese $\approx$ *1.5–2 tokens per character*

Context Window Limits

MODEL	CONTEXT WINDOW	APPROX. TEXT LENGTH
GPT-3.5 Turbo	16,384 tokens	~12,000 words (24 pages)
GPT-4 Turbo	128,000 tokens	~96,000 words (192 pages)
Claude 3 Sonnet	200,000 tokens	~150,000 words (300 pages)
Gemini 1.5 Pro	2,000,000 tokens	~1.5 million words
Llama 3 (70B)	8,192 tokens	~6,000 words (12 pages)

2.4 What Are Embeddings?



Embeddings position words in a multi-dimensional "meaning space" where similar concepts cluster together.

Embeddings are **numerical representations (vectors)** of text, images, or other data that capture semantic meaning. While tokens are the input format for LLMs, embeddings are the internal representation that allows LLMs to "understand" relationships between words.

 CITY MAP ANALOGY

Imagine a map of a city where *similar things are close together* (hospitals near hospitals, schools near libraries, beach far from mountain). Embeddings do the same for words: *"King"* and *"Queen"* are close together. *"Happy"* is near *"Joyful"* but far from *"Sad"*.

Visualizing Similarity with Cosine Scores

Cosine Similarity — "Cat" vs. Other Words

"Kitten"	0.92
"Feline"	0.85
"Dog"	0.58
"Car"	0.34
"Galaxy"	0.12

Embedding Models Comparison

MODEL	DIMENSIONS	BEST FOR
OpenAI text-embedding-3-small	1,536	General purpose
OpenAI text-embedding-3-large	3,072	High accuracy
Cohere embed-english-v3	1,024	Search + RAG
all-MiniLM-L6-v2	384	Fast, lightweight

MODEL	DIMENSIONS	BEST FOR
BGE-large	1,024	Open source, high quality

2.8 Practical Applications of Embeddings



Semantic Search (RAG Core)

User searches "car" → finds documents with "automobile", "vehicle", "sedan" even without exact keyword match.



Recommendation Systems

User liked "Inception" → Find movies with similar embeddings → Recommend "Interstellar", "The Matrix".



Clustering & Categorization

Embed customer support tickets → Cluster similar issues → Identify common problems automatically.



Anomaly Detection

If a new data point is far from the centroid (low cosine similarity) → Flag as anomaly. Great for fraud detection.

2.9 Code-Free Walkthrough: Simple Q&A System

How a company FAQ system works using embeddings (simplified to 3D for illustration):

1

Prepare Documents

Store FAQ entries: Q1 "How many vacation days?" → 20/year · Q2 "401(k) match?" → 50% up to 6% · Q3 "Reset password?" → Portal → Forgot password

2

Create Embeddings

Q1: [0.8, 0.1, 0.2] · Q2: [0.1, 0.9, 0.1] · Q3: [0.1, 0.1, 0.9]

3

User Asks "How much PTO do I get?"

Embed query → [0.75, 0.15, 0.2] . Calculate cosine similarity: Q1 = 0.96, Q2 = 0.18, Q3 = 0.21

4

Return Best Match

Q1 is most similar → Return: *"Employees get 20 vacation days per year"*.
The model understood "PTO" = "vacation days" without explicit keyword matching.

SECTION 03

Prompt Engineering

The art and science of crafting inputs that reliably produce the outputs you need — the most practical skill for any developer working with LLMs.



3.1 What Is Prompt Engineering?

Designing and optimizing input prompts to get desired outputs from LLMs.
Both an art and a science.

Prompt engineering is both an art and a science — knowing how to structure instructions, provide examples, and guide the model's reasoning.

The Anatomy of a Good Prompt

COMPONENT	DESCRIPTION	EXAMPLE
Instruction	What you want the model to do	"Summarize the following text"
Context	Additional information or documents	"Here is the article: [text]"
Output Format	How you want the answer	"Use bullet points, max 3 sentences"
Constraints	Limits or rules	"Only use info from the text"
Examples	Demonstrations of desired output	"For example: Input X → Output Y"

3.2 Zero-Shot Prompting

Give the LLM a task **without any examples**. The model relies entirely on its training.

❌ VAGUE PROMPT

"Tell me about cats" → 500-word generic essay about anything

✅ BETTER PROMPT

"Explain cat behavior in 3 bullet points for a 10-year-old" → Perfect, scoped output

ZERO-SHOT EXAMPLE

```
// Sentiment Analysis – Zero-Shot Prompt: "Classify the sentiment of this review as positive, negative, or neutral: 'The battery life is terrible, but the screen is gorgeous.'" Output: "Neutral (mixed positive and negative)"
```

3.3 Few-Shot Prompting

Provide the LLM with **examples of the desired input-output pattern** before asking it to perform the task.

FEW-SHOT TEMPLATE

```
[INSTRUCTION] Convert customer feedback into structured JSON. Example 1: Input: 'The app crashes every time I try to upload a photo.' Output: {'sentiment': 'negative', 'issue': 'crash on upload', 'urgency': 'high'} Example 2: Input: 'Love the dark mode, but search is still slow.' Output: {'sentiment': 'mixed', 'issue': 'search performance', 'urgency': 'medium'} Now process: Input: 'The battery dies within 2 hours. This is unacceptable.' Output: ← Model fills this in
```

NUMBER OF EXAMPLES	BEST FOR	TRADE-OFF
0 (Zero-shot)	Common tasks the model already knows	Fastest, cheapest
1 (One-shot)	Very simple, unambiguous tasks	May still be inconsistent
3-5 (Few-shot)	Most tasks	Good balance of cost & accuracy
10+	Complex formatting, rare patterns	Expensive (uses many tokens)

3.4 Chain-of-Thought (CoT) Prompting

Encourage the LLM to show its reasoning step-by-step before giving the final answer. This dramatically improves performance on math, logic, and multi-step problems.

✗ WITHOUT COT

Prompt: "A bat and ball cost \$1.10. Bat costs \$1.00 more. How much is the ball?"

Output: "\$0.10" ← Wrong!
(Classic cognitive bias trap)

✓ WITH COT ("LET'S THINK STEP BY STEP")

Let ball = x. Bat = x + 1.00.
x + (x + 1.00) = 1.10
2x = 0.10, x = 0.05
Answer: \$0.05 ✓

🔮 MAGIC PHRASES THAT TRIGGER COT

"Let's think step by step" · "Let's work this out logically" · "First, let me understand the problem" · "I'll solve this step by step"

3.5 Comparing Prompting Techniques



Zero-Shot

No examples, just the instruction. Model relies entirely on training knowledge.

Low Token Cost



Few-Shot

Provide 1–10 examples. Best for format control and teaching new patterns.

Medium Cost



Chain-of-Thought

Show reasoning steps. Best for math, logic, and multi-step complex tasks.

High Accuracy

3.6 Pitfalls to Avoid

PITFALL	EXPLANATION	FIX
Over-prompting	Too much irrelevant information	Keep prompts concise
Under-prompting	Missing critical instructions	Add examples or constraints
Assuming Knowledge	Model may not know recent events	Add context or use RAG
No Error Handling	Model may refuse or hallucinate	Add "If uncertain, say so"
Prompt Injection	User input overrides instructions	Sanitize input, use delimiters

3.7 Real-World Prompt Example — Code Documentation

PYTHON • FEW-SHOT

```
def calculate_discount(price, customer_type, loyalty_years):  
    if customer_type == "premium": discount = 0.20 elif loyalty_years > 5:  
    discount = 0.15 else: discount = 0.10 return price * (1 - discount) #  
    Few-shot prompt → LLM generates Google-format docstring: """ Calculate  
    discounted price based on customer type and loyalty. Args: price  
    (float): Original price before discount customer_type (str): Either  
    'premium' or 'standard' loyalty_years (int): Number of years as customer  
    Returns: float: Price after applying discount - Premium customers: 20%  
    off - Loyalty >5 years: 15% off - Others: 10% off """
```

SUMMARY

Key Takeaways



Generative AI

AI that creates new content — text, images, code, audio, video.



LLMs

Models trained on massive text to predict and generate language.



Tokens

Smallest text units (~¾ word in English). Drive cost & context limits.



Embeddings

Numerical vectors representing semantic meaning in high-dimensional space.



Zero-Shot



Few-Shot

Prompt with no examples — best for simple, well-known tasks.

Provide examples — best for format control and custom patterns.



Chain-of-Thought

Show reasoning steps — best for math, logic, and complex problems.



Vectors

Lists of numbers that position text in "meaning space" for comparison.

WHAT'S NEXT

After mastering Section 1, you are ready for...

SECTION 02

RAG

Retrieval-Augmented Generation — build Q&A systems over your own documents.

SECTION 03

Vector Databases

Storing and searching embeddings at scale with Pinecone, Weaviate, or pgvector.

SECTION 04

Agentic AI

Building autonomous agents with tools, memory, and multi-step reasoning.

SECTION 05

Production Deployment

Monitoring, evaluation, and scaling your AI systems in the real world.

Comprehensive Guide for Developers Transitioning to Generative AI

Document Version 1.0 · April 2026 · For Python & Java Developers